



Computer Games (2025-2026): Frank's Zoo v.2

Instructions

On these pages you find an assignment for the course *Computer Games*. This exercise concerns the writing of an artificial intelligence (AI) for a simple card game. This game is known under the name *Frank's Zoo*. Note that you should develop an AI that not only can play *Frank's Zoo* with the original cards of that game, but also other games of this type (as defined by the game's card set).

In this description: A *game* is a particular implementation of *Frank's Zoo* as defined by the card set. A *round* is one playthrough of a game, starting with dealing the cards and ending when all but one player has emptied their hand, and points are awarded. A *turn* is a player's opportunity to play some cards from their hand, or to pass.

When everybody has submitted their AI, we will run competitions on several games, to see which AI is most effective. A competition will be a simulation of at least 1000 rounds, but it will probably be more (depending on how much time the competition needs). The aim is to run 100,000 rounds per game if possible. It is therefore necessary that your AI runs pretty fast. An AI that is slow will be disqualified. To assess whether an AI is fast enough: your AI should use less than 0.01 seconds per round. If you test it for different numbers of rounds, you can estimate the time complexity of your AI. It should be linear (or better) with respect to the number of rounds. If your AI needs some setup time when it is created (during the execution of `__init__()`), you may use up to 10 seconds for that.

As indicated, a competition will be run for different games, i.e., different card sets. One of these sets will be the original *Frank's Zoo* card set. However, other card sets will also be used (two more are delivered with this assignment, but the instructor may add one or two extra ones during the competition, unknown at present). Your AI should be able to play all of them. Preferably, it is able to handle any card set equally well.

The maximum number of points that you can score for this exercise is 10. To get a high number of points, make sure that you implement an interesting algorithm, and write understandable code. An AI that makes the competition crash is disqualified, so test well. Please make sure that your AI does not produce any printed output. There is no need to write “winning” code (though it cannot hurt, of course). If you create several different AIs, submit only one for the competition, but you can also submit the other AIs to the instructor, explaining why you prefer the one you submitted over the others (probably because it seems to be the strongest). If you submit an AI that can play the original *Frank's Zoo* card set well, but not any other ones very well, that is acceptable but not preferred. An adaptive AI would be quite interesting for this assignment.

Your AI should consist of a single Python file. You can make use of standard Python libraries, but in principle not anything else. If you want to use something beyond the standards, first contact the instructor. In the past some students wanted to use *numpy* or *pandas* – such libraries are not allowed because: (1) they make the competition too slow, and (2) what they do can easily be done with regular Python code.

When you have completed your AI, submit it via Canvas. Make sure that you have included your name and student number in your AI's header. The deadline for submission is the Friday before the last lecture. Your submission should be accompanied by a short report (1-2 pages suffices, please use a PDF or Word file) which explains your tactical approach to the game, different approaches you may have considered, and how you tested. If you used particular tools to help the development (including AI tools for coding help), or inspiration from literature, please list that also in your report. It is allowed, but needs to be acknowledged.

Frank's Zoo

Frank's Zoo is a fairly simple card game for four to six players (we will use four players in our competition). It uses a deck of 60 cards, each depicting an animal. For each animal, the card also identifies its predators. The cards are listed in Table 1.

Cards are dealt evenly among the players. The player left of the dealer opens a round, and players get turns going clockwise around the table. When a player gets a turn, they can do one of the following:

- Pass, i.e., skip a turn.
- Move: play one or more cards to defeat the last move that is on the table.
- When opening: play any legal move (there is no move yet to defeat).

When, after a move of one player, all other players pass, the player who played the last move gets to open anew. If this player has already emptied their hand, the opening move goes to the player to their left. The goal of the game is to empty one's hand as quickly as possible.

Animal	#cards	Predators	ID
Whale	5	None	12
Elephant	5	Mouse	11
Crocodile	5	Elephant	10
Polar Bear	5	Whale, Elephant	9
Lion	5	Elephant	8
Seal	5	Whale, Polar Bear	7
Fox	5	Elephant, Crocodile, Polar Bear, Lion	6
Perch	5	Whale, Crocodile, Polar Bear, Seal	5
Hedgehog	5	Fox	4
Fish	5	Whale, Crocodile, Seal, Perch	3
Mouse	5	Crocodile, Polar Bear, Lion, Seal, Fox, Hedgehog	2
Mosquito	4	Hedgehog, Fish, Mouse	1
Chameleon	1	special (Joker)	0

Table 1: Cards in *Frank's Zoo*.

A legal move is one that consists of cards of one type of animal. This should be:

- Either: A predator for the animal of the previous move. The number of cards should be exactly the same number as there were in the previous move;
- Or: The same animal as the previous move. The number of cards should be exactly one more than the number of cards that were in the previous move.

The Chameleon can be used to replace one of the cards in a move, as long as the move contains at least one “normal” animal card. Furthermore, in a move that contains one or more Elephants, at most one Mosquito can be added, which will then “turn into an Elephant”. Note that when you play two cards, namely a Mosquito and the Chameleon, the Chameleon will be a Mosquito. If you want the Chameleon to be an Elephant, there must be at least one Elephant in the play.

Examples:

One Hedgehog gets defeated by:

- One Fox
- Two Hedgehogs
- One Hedgehog and one Chameleon

Note that these are the only possibilities. You cannot play, for instance, two Foxes or three Hedgehogs.

One Elephant gets defeated by:

- One Mouse
- Two Elephants
- One Elephant and one Chameleon
- One Elephant and one Mosquito

Note that one Elephant does *not* get defeated by one Chameleon, because a Chameleon cannot be played on its own; it is not a Mouse if it is played on its own. Moreover, it can also not be defeated by a Chameleon and a Mosquito, because in that case the Chameleon becomes a Mosquito, and two Mosquitos do not defeat an Elephant.

Three Crocodiles get defeated by:

- Three Elephants
- Two Elephants and one Chameleon
- Two Elephants and one Mosquito
- One Elephant, one Mosquito, and one Chameleon

Theoretically, four Crocodiles would also defeat three Crocodiles, but due to the fact that there are only five Crocodiles total in the deck, that can never happen.

Two Polar Bears get defeated by:

- Two Elephants
- One Elephant and one Chameleon
- One Elephant and one Mosquito
- Two Whales
- One Whale and one Chameleon
- Three Polar Bears
- Two Polar Bears and one Chameleon

This move is not defeated by an Elephant plus a Whale, since a move cannot combine two different kinds of animals. The exception for the previous statement is that one Mosquito may be added to Elephants, as it then “turns into an Elephant.”

When a player empties their hand, they are out of the round. The first player who empties their hand gets a number of points equal to the number of players (in our case, 4). The second player to do so gets one point less, the third player again one point less, etc. When everyone but one player is out, the last player receives zero points. The game is usually played over multiple rounds, and the players accumulate their points over the rounds, the player with the highest end total being the final winner.

Alternative Card Sets

Card sets are delivered to the game engine using a file. These files are text files that are formatted as follows (this description is only useful to you if you want to look into these files; they are loaded automatically by the engine):

- The first line is the word **ANIMAL** (in capitals).
- This is followed by lines that specify the individual cards, as three items separated by comma's, namely the name of the card, the ID of the card, and the number of times the card is in the deck. For instance: `mosquito,1,4` indicates that the card named “mosquito” has ID 1 and is 4 times in the deck. Note that ID 0 is reserved for a Joker (which is not necessarily in the deck, but if the deck contains Jokers, they have ID 0). By definition, Jokers can never be played on their own.
- After the specification of the cards, follows a line with the word **PREDATOR** (in capitals).
- This is followed by lines that specify which animals a given animal is the predator of, namely as follows: the first item on the line is the ID of the predator, which is followed by a colon, which is followed by the IDs of the preys of the predator, separated by comma's. For instance: `2:1,11` indicates that the animal with ID 2 (which is the Mouse in the *Frank's Zoo* deck) is the predator of the Mosquito (ID 1) and the Elephant (ID 11).

- After the specification of the predators, follows a line with the word `SUBSTITUTE` (in capitals).
- This is followed by zero or more lines, which specify substitutes for cards, as follows: the first item on the line is the ID of a card, which is followed by a colon, which is followed by the IDs of the cards which may be substituted for the first item, separated by comma's. For instance: `11:0,1` indicates that the animal with ID 11 (the Elephant) may be substituted by the animals with ID 0 (the Chameleon) or ID 1 (the Mosquito). Note that it is given in the rules that in a play, you may include a particular substitute only once (e.g., you may accompany an Elephant by a Mosquito, but not by two Mosquitos).

Included with the engine are three files with card definitions. `fzdefault.txt` is the original *Frank's Zoo* deck. `fzsimplified.txt` is a simplified version of the *Frank's Zoo* deck, namely without substitutions, no Chameleon, and five Mosquitos. `fzstratego.txt` is a new deck inspired by the game of *Stratego*. It contains 40 cards total, with fairly simple relationships, but different counts for the card types (e.g., it contains 8 “scouts” but only 1 “general”).

Running a Competition

To run a competition, start the file `FranksZoo.py`. If you do that from the command line, you can give command line parameters:

```
python -u FranksZoo.py 1000 cardfile aifile1 aifile2 ...
```

The first command line parameter (1000 in the example above) is the number of rounds that should be played in the competition. This is followed by the name of a file that contains a definition of the cards used. After that you can specify a list of python files that contain AIs that you want to enter into the competition. If you do not specify any command line parameters, in `FranksZoo.py` default values are specified.

If you want to run the competition from the editor (which is probably easiest), you can (temporarily) change the values for `DEFAULTLENGTH`, `DEFAULTDECK` and `DEFAULTAIS` at the top of the `FranksZoo.py` file. `DEFAULTLENGTH` indicates how many rounds you want to run, `DEFAULTDECK` indicates the name of the file which contains the card definitions (if empty, the original *Frank's Zoo* cards will be used), and `DEFAULTAIS` is a list of all the AI files in the competition.

If you change `DEFAULTAIS`, for instance, to `['FranksZooBeginners', 'MyAI']`, it will load all the AIs from `FranksZooBeginners.py` and from `MyAI.py`. You can also simply run the `FranksZoo.py` file with an empty list for `DEFAULTAIS`. In that case, the engine will scan the current folder for all Python files which contain AIs, and will include those AIs in the competition. This means that you should only have Python files with AIs that you want to include in the current folder; move all other AIs to a different folder. In particular, you probably do not want to include the “self-play” AI (`FranksZooSelfPlay.py`).

If there are more than four AIs in the competition, every turn a random selection of four AIs will be made. This entails that not every AI may play the same number of rounds, and configurations of the selected players (e.g., who plays first) will also not

be consistent. Of course, the shuffling of the deck will also result in variations. Statistically, if the number of games is large enough, the best AI of the pool should rise to the top anyway.

At the end of the competition, the game prints an ordered list of all the AIs that entered, with a number that indicates in how many games they were, and their average score (a floating-point value between 0 and 4). The higher the average score, the better it is. It will also show the total number of seconds that the AI used, and the number of seconds per game. Try to make sure that the total number of seconds per game is about the same for your AI, regardless of the number of matches you run. This would indicate that your AI is no worse than linear time complexity.

Classes

Before I discuss how to write an AI for *Frank's Zoo* in Python, I want to bring up the most important Python classes that you will use. These classes are found in `FranksZooGame.py` and `FranksZooState.py`.

A card of the game is represented by the class `Animal`. Instances of this class have the following attributes: `name`, `id`, `number`, `predator`, `prey`, `transform`, `substitute`, and `ordering`. `name` is the animal's name, but the best way to refer to an animal is by `id`, which is an integer (the IDs for the *Frank's Zoo* deck are included in Table 1; note that ID 0 is reserved for the Joker). `number` indicates how often the card occurs in the deck. `predator` is a list of the animals which are predators for the animal, and `prey` is a list of animals which are preys of the animal. `transform` is a list of the animals into which the animal can transform (a Mosquito can transform into an Elephant), and `substitute` is a list of animals that can be substitutes for the animal. `ordering` is used to order the animals in a hand of cards (which is equal to the ID, so it is superfluous but included for future use).

A set of cards is represented by a `Hand`. Instances of `Hand` have only one attribute, which is a list of cards (`Animals`) in the attribute `cards`.

The class `Play` inherits from `Hand`. This is the class that the engine will use to communicate with AIs. A `Play` is a set of cards in the attribute `cards` (from `Hand`), and a few more attributes, namely the following: the name of the player who made this play in the attribute `player`, and two booleans, `opening` and `out`, which indicate whether it was an opening play, and whether it was a play which made the player empty their hand. When an AI creates a `Play` object, only the set of cards should be filled in. The other attributes are filled in by the main program.

In the file `FranksZooState.py` a `State` is defined. This is used to communicate to the AI the current state of the game. It contains an attribute `history`, which is a list of all the plays (as `Play` objects) made in the round until now. It also contains a list `players`, which are instances of the class `PlayerState`. The players are ordered in the sequence that they take turns, starting with the player who got to start the round. For each player there is an attribute `name` which contains the player name, an attribute `score` which contains the average score the player achieved over all the

rounds played until now, and an attribute `handsize` which contains how many cards the player still has in their hand now.

The last important class that you should be aware of is `Player`, which is defined in `FranksZooPlayer.py`. A `Player` is an AI which plays the game.

How to write an AI

To write an AI, create a new Python file, and create a new class in it. This class should inherit its features from `Player`, which is defined in `FranksZooPlayer.py`. You should also create an `__init__()` method, which gets a `cardlist` as parameter. The `__init__()` method should make a `super()` call to the `__init__()` method from `Player`, with as parameters the `cardlist` that was supplied, and the name that you give the AI (a string).

So, suppose that your new AI is called `MyAI`, then you will write in your file the following code:

```
from FranksZooPlayer import Player

class MyAI(Player):
    def __init__( self, cardlist ):
        super().__init__( cardlist, 'MyAI' )
```

`cardlist` is a `Hand` object, of which the attribute `cards` contains a copy of each type of card (`Animal`) in the deck. You can use this if you want to do some kind of analysis of the deck when your AI is created. This is optional, of course. Note that you are not allowed to make changes to this parameter, as it will be given to all AIs.

What you have to do is create at least one method, namely `play()`. `play()` is used to take a turn. It should return a valid play in the form of a `Play` object.

`play()` is defined as follows:

```
def play( self, lastplay, possible, state ):
```

`lastplay` is an object of the class `Play`, which represents the `Play` to which the AI must answer. If the AI must make an opening play, `lastplay` is `None`.

`possible` is a list of `Play` objects which are all the possible valid plays that the `play()` method can return. The first is always representing “pass,” which is an empty list of cards, except when this is an opening play, in which case “pass” is not allowed – the only exception is that there would be only one card in the hand of the player, namely the Chameleon, as a Chameleon cannot be played on its own. In that case, and that case alone, a player may “pass” on an opening.

`state` represents the current state of the game as a `State` object.

As for the return value of `play()`: `play()` must return a valid `Play` object, i.e., one of the `Plays` in the list `possible`. This can be done by returning one of the

objects in `possible`, or creating a `Play` object in the method itself. If you create your own `Play` object, you only should fill in the cards. The other attributes in the `Play` object will be filled in by the main engine. When you return a `Play` object, you do not need to remove the cards from your hand – the main engine will do that. You only need to say “this is what I want to play,” and the rest will be handled automatically.

If you return a `Play` object which is not valid, the engine makes a random play for you. This will also print an error message.

`play()` is called any time a player must take a turn. Even when there is only one possible play that can be made (i.e., `possible` contains only one object), it is still called.

Note that the method `play()` is not allowed to make changes to the arguments passed to the method. These are just meant as information. They are used by other methods as well. Naturally, I could avoid the potential problem of an AI making changes to these arguments by making copies of them before passing them to the `play()` method, but such copying tends to be rather time-intensive, and I don’t want to slow down the process. Besides, if the programmer really wants they can interfere with any object in the program, but that would be cheating, and cheating is not allowed anyway.

There are two more methods that can be used. The `startgame()` method is called at the start of the round, and the `endgame()` method at the end of the round. `startgame()` can be used to make some initializations, while `endgame()` can be used to track statistics. `endgame()` gets the current game state as argument.

Cheating

If you want to cheat in this game, of course that is possible. This is Python. Everything is visible, everything is accessible. Cheating is, in fact, so easy that I have not taken the trouble of trying to prevent it in code. Instead, I am going to tell you straight out: you are not allowed to cheat. If you cheat, you are disqualified. I will definitely examine everybody’s code, with special attention to the code of those who score well.

What is your AI allowed to do? You may look at the arguments given to the `play()` method. You may *not* make changes to them. Other than making changes to attributes that you create for your own AI, you are only allowed to effectuate changes by returning plays from the `play()` method. Other changes are not allowed. You are not allowed to look at information that is secret to you during the game, i.e., the cards that the other players have in hand.

Testing

To test the capabilities of your AI, you will probably first try it out against the AIs in `FranksZooBeginners.py`. Know, however, that these AIs are rather weak. You should be able to create an AI that plays a better game.

If you want to play the game yourself against AIs, include the AI that I placed in `FranksZooSelfPlay.py`. Make sure that there are only three opponents, and set the number of games to 1.

You might use an approach where you create several different AIs, and try to improve them while testing them against earlier incarnations of your AIs. You might also test your AI against those of your fellow students, if they are willing to engage in such activities. I encourage you to do that. For those who like a real challenge, implementing an adaptive AI that learns better behaviour over time might be a cool project (even if the results are meagre, I would really appreciate the attempt).

During your tests, you may want to print out certain objects, such as the `State` objects. Some have suitable `__str__` methods encoded. However, make sure that you remove those prints again before submitting your final code.

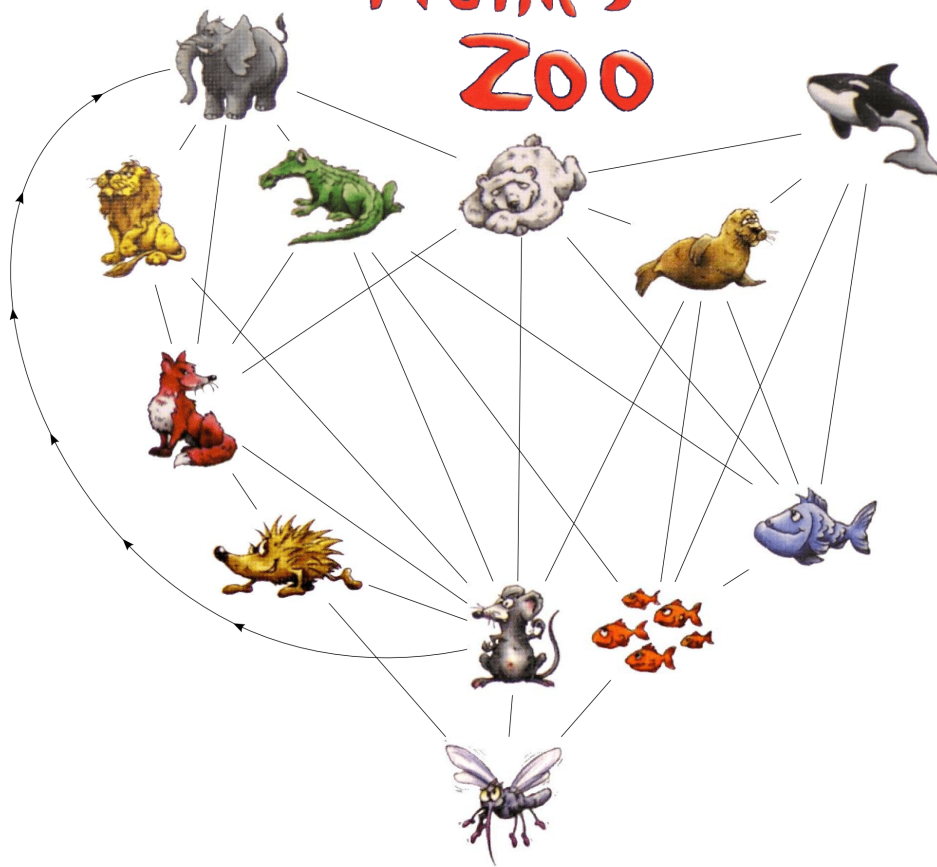
Competition Time

I will run live competitions during the last lecture. You are encouraged to be there. I would like those who score well or did something really interesting to explain briefly how they approached the AI to the other students (there is no need to prepare this).

Further Information

If you run into problems with the code or have questions, please contact the instructor. The code was created last year and significantly updated this year, and is quite complex, so I cannot guarantee that there are no errors in it.

Frank's Zoo



Who outranks who?	Whale	Elephant	Crocodile	Polar bear	Lion	Seal	Fox	Perch	Hedgehog	Fish	Mouse	Mosquito
Whale	>											
Elephant		>										
Crocodile			>									
Polar bear				>								
Lion					>							
Seal						>						
Fox							>					
Perch								>				
Hedgehog									>			
Fish										>		
Mouse											>	
Mosquito												>

To read the table, choose the row of the animal you want. The animals that outrank that animal are shown by an  in that row. Thus, the elephant is outranked only by the mouse.